LEPAn app

CONTENTS

"Main Page"

"Single Corpus Analysis"

"Compare Corpora"

"Explore Corpora"

Requirements

streamlit

pandas

numpy

wordcloud

plotly

matplotlib

seaborn

sklearn

openpyxl

PAGES

"Main Page"

Show info about the project, render an abstract of the Trust paper. No specific function is executed here.

Used components:

st.expander()

st.container()

"Single Corpus Analysis"

"Analytics Unit" : "Text-Based Analysis" (default)

"Analytics Module" : "LEP Distribution" (default)

Show distribution of logos, ethos and pathos in the selected corpora.

Used components:

st.multiselect()

st.columns()

st.radio()

st.tabs()

st.pyplot()

Used functions:

plot_pathos_emo

plot_pathos_emo_counts

plot_rhetoric_basic_stats[1/ 2/ 3]_post

plot_rhetoric_basic_stats[1/ 2/ 3]

"Analytics Module" : "WordCloud"

Visualise words with high precision for logos, ethos or pathos.

Used components:

st.selectbox()

st.radio()

st.pyplot()

st.info()

st.slider()

Used functions:

prepare_cloud_lexeme_data

wordcloud_lexeme

make_word_cloud

"Analytics Unit" : "Person-Based Analysis"

"Analytics Module" : "(Anti)Heroes" (default)

Show the (anti)hero metrics for the chosen target entity. Metrics based on the ethos annotation – what percentage of people view him or her as hero and anti-hero (that is, support or attack this person in terms of ethos).

Content displayed directly on the page – no special functions needed.

Used components:

st.selectbox()

st.container()

st.columns()

st.metric()

st.pyplot()

st.info()

st.expander()

"Analytics Module" : "LEP Strategies"

Show the distribution in form of a histogram or heatmap.

Used components:

st.radio()

st.pyplot()

st.metric()

st.columns()

st.expander()

Used functions:

user_stats_app

"Analytics Module" : "LEP Behaviour"

Show the distribution of metric in form of a histogram (or seaborn *countplot*).

Used components:

st.radio()

st.pyplot()

st.selectbox()

st.expander()

Used functions:

ethos_logos_LEPbehavior_metric

pathos_LEPbehavior_metric

"Analytics Module" : "LEP Profiles"

First, Show the distribution of LEP behaviour metric in form of a histogram (or seaborn *countplot*). Then, display the most negative and positive users based on this metric. Show examples of text generated by a chosen negative and positive user.

Used components:

st.radio()

st.pyplot()

st.selectbox()

st.columns()

st.metric()

st.container()

st.dataframe()

st.error()

st.success()

st.markdown()

st.expander()

Used functions:

user_rhetoric_v2

user_stats_app

"Compare Corpora"

"Analytics Unit" : "Text-Based Analysis" (default)

"Analytics Module" : "LEP Distribution" (default)

Show next to each other distributions of logos, ethos and pathos in the selected set of corpora.

Used components:

st.selectbox()

st.pyplot()

st.radio()

st.error()

st.success()

st.warning()

st.info()

st.dataframe()

Used functions:

plotRhetoricCompare[1/ 2/ 3]

plotRhetoricCompare[1/ 2/ 3]_finegrained

"Analytics Module" : "WordCloud"

Visualise words with high precision for logos, ethos or pathos in all chosen corpora.

Used components:

st.selectbox()

st.pyplot()

st.dataframe()

st.slider()

st.radio()

st.columns()

st.warning()

Used functions:

prepare_cloud_lexeme_data

wordcloud_lexeme_compare2

make_word_cloud

"Analytics Unit" : "Person-Based Analysis"

"Analytics Module" : "(Anti)Heroes" (default)

Show the distribution of (anti)heroes in the selected set of corpora. Metric based on the ethos annotation – what percentage of people view him or her as hero and anti-hero (that is, support or attack this person in terms of ethos). An entity can be either a hero or antihero – by subtracting antihero score from the hero score. Visualise names and scores as barplot.

Content displayed directly on the page – no special functions needed.

Used components:

st.warning()

st.pyplot()

st.columns()

st.metric()

"Explore Corpora"

Allows to explore a chosen dataset. All relevant columns from the dataset are depicted in a multiselect list. A user can choose columns and then specify values for them in order to define conditions for displaying only certain cases (examples) from the dataset. No specific function is executed here.

Used components:

st.selectbox()

st.dataframe()

st.columns()

st.multiselect()

FUNCTIONS

How to read the docs:

FunctionName(function arguments : type of input):

Short description of the function.

Full source code:

– actual implementation of a function in *Python* and *streamlit*

---

style_css(path_to_file: str):

Opens and loads the css file to load the style of the interface.

Full source code:

```
with open(path_to_file) as f:
    st.markdown(f"<style>{f.read()}</style>", unsafe_allow_html=True)
```

@cache

load_dataset(dataset_name: str):

It use pandas *read_excel()* or *read_csv()* to load data and cache the result with *streamlit* so the data need not to be reloaded in every call. Nonetheless, data can be loaded without this function simply with pandas *read_excel()*.

Full source code:

```
if dataset_name == "US-Presidential-2016_Reddit":
    data = pd.read_excel(r"C:\Users\user1\Downloads\app_US2016.xlsx", index_col = 0)
elif dataset_name ==  "Conspiracy-Theories-Vaccines-2021_Reddit":
    data = pd.read_excel(r"C:\Users\user1\Downloads\app_conspiracy.xlsx", index_col = 0)
return data
```

user_rhetoric_v2(dataframe: dataframe, source_column: str = "Source" , ethos_column: str = "ethos_name", pathos_column: str = "pathos_name", logos_column: str = "logos_name")

Calculates LEP behaviour ("rhetoric_metric") for each user from the dataframe based on logos, ethos and pathos annotation. First, the function maps weights for each category. Then, it multiplies the weights and number of texts from each category, and sums everything together (logos, ethos, and pathos categories). So, for example it maps ethos attacks to its weight, i.e. –1 and then multiplies –1 by the number of texts generated by a user *u* classified as ethos attacks in the annotation. Then, the same for ethos supports, pathos negative, etc. Then, it sums all the obtained values from each category (of ethos attacks, ethos supports, pathos negative, etc).

Full source code:

```
map_ethos_weight = {
        "attack":-1,
        "neutral":0,
        "support":1      }
 map_pathos_weight = {
        "negative":-1,
        "neutral":0,
        "positive":1      }
 map_logos_weight = {
        "attack":-0.5,
        "neutral":0,
        "support":0.5    }
 metric_value = []
 users_list = []
 for user in dataframe:
  users_list.append(user)
  df_user = dataframe[dataframe[source_column] == u]
  ethos_pathos_logos_user = 0
  df_user_rhetoric = df_user.groupby([pathos_col, logos_col, ethos_col],
                              as_index=False).size()
  # map weights
```

```
    df_user_rhetoric[pathos_col] = df_user_rhetoric[pathos_col].map(map_pathos_weight)
    df_user_rhetoric[logos_col] = df_user_rhetoric[logos_col].map(map_logos_weight)
    df_user_rhetoric[ethos_col] = df_user_rhetoric[ethos_col].map(map_ethos_weight)
    ethos_pathos_logos_sum_ids = []
    for id in df_user_rhetoric.index:
      ethos_pathos_val = np.sum(df_user_rhetoric.loc[id, str(pathos_col):str(ethos_col)].to_numpy())
      ethos_pathos_val = ethos_pathos_val * df_user_rhetoric.loc[id, "size"]
      ethos_pathos_logos_sum_ids.append(ethos_pathos_val)
    ethos_pathos_logos_user = np.sum(ethos_pathos_logos_sum_ids)
    metric_value.append(int(ethos_pathos_logos_user))
df = pd.DataFrame({"user": users_list, "rhetoric_metric": metric_value})
return df
```

user_stats_app(dataframe: dataframe, source_column: str = "Source", logos_column: str = "logos_name", ethos_column: str = "ethos_name", pathos_column: str = "pathos_name"):

Calculates the numbers and proportions of texts that belong to each category of logos, ethos and pathos for each user from the dataframe.

Full source code:

```
sources_list = dataframe[dataframe[source_column] != "nan"][source_column].unique()
df = pd.DataFrame(columns = ["user", "text_n",
                    "ethos_n", "ethos_support_n", "ethos_attack_n",
                    "pathos_n", "pathos_negative_n", "pathos_positive_n",
                    "logos_n", "logos_support_n", "logos_attack_n",
                   "ethos_percent", "ethos_support_percent", "ethos_attack_percent",
                   "pathos_percent", "pathos_negative_percent", "pathos_positive_percent",
                   "logos_percent", "logos_support_percent", "logos_attack_percent"])
users_list = []
for i, u in enumerate(sources_list):
  users_list.append(str(u))
  df_user = dataframe[dataframe[source_column] == u]
  N_user = int(len(df_user))
  df_user_logos = df_user.groupby(logos_column, as_index = False)["Text"].size()
  try:
    N_ra = int(df_user_logos[df_user_logos[logos_column] == "support"]["size"].iloc[0])
    N_ra = int(N_ra)
  except:
    N_ra = 0
  df_user_ca = df_user.groupby(logos_column, as_index = False)["Text"].size()
  try:
    N_ca = int(df_user_ca[df_user_ca[logos_column] == "attack"]["size"].iloc[0])
    N_ca = int(N_ca)
  except:
    N_ca = 0
```

```
  df_user_ethos = df_user.groupby(ethos_column, as_index = False)["Text"].size()
  try:
    N_support = int(df_user_ethos[df_user_ethos[ethos_column] == "support"]["size"].iloc[0])
  except:
    N_support = 0
  try:
    N_attack = int(df_user_ethos[df_user_ethos[ethos_column] == "attack"]["size"].iloc[0])
  except:
    N_attack=0
  df_user_pathos = df_user.groupby(pathos_column, as_index = False)["Text"].size()
  try:
    N_neg = int(df_user_pathos[df_user_pathos[pathos_column] == "negative"]["size"].iloc[0])
  except:
    N_neg = 0
  try:
    N_pos = int(df_user_pathos[df_user_pathos[pathos_column] == "positive"]["size"].iloc[0])
  except:
    N_pos = 0
  counts_list = [N_support+N_attack, N_support, N_attack, N_neg+N_pos, N_neg, N_pos,
                 N_ra+N_ca, N_ra, N_ca]
  percent_list = list((np.array(counts_list) / N_user).round(3) * 100)
  df.loc[i] = [u] + [N_user] + counts_list + percent_list
return df
```

prepare_cloud_lexeme_data(data_neutral: dataframe, data_support: dataframe, data_attack: dataframe):

Creates a dictionary of words with the number of times each word occurs in texts from each category (support, attack, neutral) of the chosen dimension (either logos, ethos, or pathos). Then, with the use of *wordcloud_lexeme()* **function it calculates a precision score for each word for the chosen category and makes a word-cloud figure with words above the specified precision threshold.**

Full source code:

```
# neutral df
 neu_text = " ".join(data_neutral["clean_Text_lemmatized"].apply(lambda x: " ".join(t for t in
                set(x.split()))).to_numpy())
 count_dict_df_neu_text = Counter(neu_text.split(" "))
 df_neu_text = pd.DataFrame( {"word": list(count_dict_df_neu_text.keys()),
                    "neutral #": list(count_dict_df_neu_text.values())} )
 df_neu_text.sort_values(by = "neutral #", inplace=True, ascending=False)
 df_neu_text.reset_index(inplace=True, drop=True)
 #df_neu_text = df_neu_text[~(df_neu_text.word.isin(stops))]

 # support df
 supp_text = " ".join(data_support["clean_Text_lemmatized"].apply(lambda x: " ".join(t for t in
                set(x.split()))).to_numpy())
 count_dict_df_supp_text = Counter(supp_text.split(" "))
 df_supp_text = pd.DataFrame( {"word": list(count_dict_df_supp_text.keys()),
                    "support #": list(count_dict_df_supp_text.values())} )

 df_supp_text.sort_values(by = "support #", inplace=True, ascending=False)
 df_supp_text.reset_index(inplace=True, drop=True)
 #df_supp_text = df_supp_text[~(df_supp_text.word.isin(stops))]
```

```
merg = pd.merge(df_supp_text, df_neu_text, on = "word", how = "outer")


#attack df
att_text = " ".join(data_attack["clean_Text_lemmatized"].apply(lambda x: " ".join(t for t in
            set(x.split()))).to_numpy())
count_dict_df_att_text = Counter(att_text.split(" "))
df_att_text = pd.DataFrame( {"word": list(count_dict_df_att_text.keys()),
                    "attack #": list(count_dict_df_att_text.values())} )


df_att_text.sort_values(by = "attack #", inplace=True, ascending=False)
df_att_text.reset_index(inplace=True, drop=True)
#df_att_text = df_att_text[~(df_att_text.word.isin(stops))]


df2 = pd.merge(merg, df_att_text, on = "word", how = "outer")
df2.fillna(0, inplace=True)
df2["general #"] = df2["support #"] + df2["attack #"] + df2["neutral #"]
df2["word"] = df2["word"].str.replace("""", "_").replace("”", "_").replace""", "_")
return df2
```

wordcloud_lexeme(dataframe: dataframe, lexeme_threshold: int = 90, analysis_for: str = "support", cmap_wordcloud: str = "crest"):

Calculates a precision score for each word for the chosen category and by calling *make_word_cloud()* **function makes a word-cloud figure with words above the specified precision threshold.**

Dataframe used in the function must come from the *prepare_cloud_lexeme_data()* **function.**

Full source code:

```
if analysis_for == "attack":
   #print(f"Analysis for: {analysis_for} ")
   cmap_wordcloud = "gist_heat"
   dataframe["% lexeme"] = (round(dataframe["attack #"] / dataframe["general #"], 3) *
                                    100).apply(float) # att
 elif analysis_for == "both":
   #print(f"Analysis for: {analysis_for} ")
   cmap_wordcloud = "viridis"
   dataframe["% lexeme"] = (round((dataframe["support #"] + dataframe["attack #"]) /
                                    dataframe["general #"], 3) * 100).apply(float) # both supp & att
 else:
   #print(f"Analysis for: {analysis_for} ")
   dataframe["% lexeme"] = (round(dataframe["support #"] / dataframe["general #"], 3) *
                                     100).apply(float) # supp
 dfcloud = dataframe[(dataframe["% lexeme"] >= int(lexeme_threshold)) & (dataframe["general #"] >
              1) & (dataframe.word.map(len)>3)]
 text = []
 for i in dfcloud.index:
   w = dfcloud.loc[i, "word"]
   w = str(w).strip()
   if analysis_for == "both":
     n = int(dfcloud.loc[i, "support #"] + dfcloud.loc[i, "attack #"])
   else:
```

```
    n = int(dfcloud.loc[i, str(analysis_for)+" #"]) #  + dfcloud.loc[i, "attack #"]   dfcloud.loc[i, "support
          #"]+  general
  l = np.repeat(w, n)
  text.extend(l)
import random
random.shuffle(text)
figure_cloud = make_word_cloud(" ".join(text), 1000, 620, "#1E1E1E", str(cmap_wordcloud))
                    #gist_heat / flare_r crest viridis
return figure_cloud
```

make_word_cloud(comment_words: str, width: int = 1100, height: int = 650, colour: str = "black", colormap: str = "brg"):

It makes a word-cloud figure from text in comment_words.

Full source code:

```
stopwords = set(STOPWORDS)
    wordcloud = WordCloud(collocations=False, max_words=250, colormap=colormap, width = width,
                        height = height,
            background_color ="white",
            min_font_size = 14, stopwords = stopwords).generate(comment_words)
    fig, ax = plt.subplots(figsize = (width/ 100, height/100), facecolor = colour)
    ax.imshow(wordcloud)
    plt.axis("off")
    plt.tight_layout(pad = 0)
    plt.show()
    return fig
```

wordcloud_lexeme_compare2(dataframes: list, lexeme_threshold: int = 90, analysis_for: str = "support", cmap_wordcloud: str = "crest"):

For each dataframe in the *dataframes* (list) creates a dictionary for the specified category. Then, it computes the precision score for each word for the chosen category.

Words above the precision threshold value in each dataframe are visualised in the word-cloud.

Full source code:

```
if analysis_for == "attack":
    #print(f"Analysis for: {analysis_for} ")
    cmap_wordcloud = "gist_heat"
    for dataframe in dataframes:
        dataframe["% lexeme"] = (round(dataframe["attack #"] / dataframe["general #"], 3) *
                                 100).apply(float) # att
  elif analysis_for == "both":
    #print(f"Analysis for: {analysis_for} ")
    cmap_wordcloud = "viridis"
    for dataframe in dataframes:
        dataframe["% lexeme"] = (round((dataframe["support #"] + dataframe["attack #"]) /
                                 dataframe["general #"], 3) * 100).apply(float) # both supp & att
  else:
    #print(f"Analysis for: {analysis_for} ")
    for dataframe in dataframes:
        dataframe["% lexeme"] = (round(dataframe["support #"] / dataframe["general #"], 3) *
                                 100).apply(float) # supp
  dfcloud = pd.DataFrame(columns = dataframes[0].columns)
  dfcloud["corpora_n"] = np.nan
  list_uniq_words = []
  dict_uniq_words = {}
  dict_corpora_name = {}
  for n, dataframe in enumerate(dataframes):
      dict_corpora_name[n] = dataframe["Dataset"].iloc[0]
```

```
        dfcloud1 = dataframe[(dataframe["% lexeme"] >= int(lexeme_threshold)) &
(dataframe["general #"] > 1) & (dataframe.word.map(len)>3)]
        dfcloud1["corpora_n"] = int(n)
        dfcloud1["word"] = dfcloud1["word"].apply(lambda x: str(x).strip())
        list_uniq_words.extend(dfcloud1["word"].unique())
        dict_uniq_words[str(n)] = set(dfcloud1["word"].unique())
        dfcloud = pd.concat([dfcloud, dfcloud1], axis = 0)
    dfcloud = dfcloud.drop_duplicates()
    dfcloud = dfcloud.reset_index(drop=True)
    list_uniq_words = set(list_uniq_words)
    list_uniq_words_select = []
    for w in list_uniq_words:
        shared_word = []
        for n in range(len(dataframes)):
            if w in dfcloud[dfcloud["corpora_n"] == int(n)]["word"].unique():
                shared_word.append(1)
            else:
                shared_word.append(0)
        if np.sum(shared_word) == len(dataframes):
            list_uniq_words_select.append(w)
    dfcloud = dfcloud[dfcloud.word.isin(list_uniq_words_select)]
    n_words = dfcloud["word"].nunique()
    text = []
    for w in dfcloud["word"].unique():
      w = str(w).strip()
      if analysis_for == "both":
          ids = dfcloud[dfcloud.word == w].index
          n = 0
          for i in ids:
              nn = int(dfcloud.loc[i, "support #"] + dfcloud.loc[i, "attack #"])
              n += nn
      else:
          ids = dfcloud[dfcloud.word == w].index
```

```
        n = 0
        for i in ids:
            nn = dfcloud.loc[i, str(analysis_for)+" #"] #  + dfcloud.loc[i, "attack #"]   dfcloud.loc[i,
"support
                #"]+  general
            n += int(nn)
      l = np.repeat(w, n)
      text.extend(l)
    dfcloud = dfcloud.sort_values(by = ["corpora_n", "word", "% lexeme"])
    dfcloud = dfcloud.drop(["corpora_n"], axis=1)
    dfcloud = dfcloud.rename(columns={"Dataset": "corpora", "% lexeme": "precision", "general #":
                "overall #"})
    import random
    random.shuffle(text)
    st.write(f"There are {n_words} shared words.")
    figure_cloud = make_word_cloud(" ".join(text), 800, 500, "#1E1E1E", str(cmap_wordcloud))
                        #gist_heat / flare_r crest viridis
    return figure_cloud, dfcloud
```

plot_pathos_emo(dataframe: dataframe):

Make a barplot with proportions (percentages) of basic emotions.

Full source code:

```
df_melt_emo_pathos = dataframe[pathos_cols[4:]].fillna(0).melt(var_name="emotion", value_name
= "value")

  df_melt_emo_pathos_perc =

                              df_melt_emo_pathos.groupby("emotion")["value"].mean().reset_index()

  df_melt_emo_pathos_perc["value"] = df_melt_emo_pathos_perc["value"].round(3) * 100


  color_map_pathos_emo = {"anger": "#BB0000","anticipation": "#D87D00","disgust": "#BB0000",

                                  "fear":"#BB0000","happiness": "#026F00","sadness":
                              "#BB0000","surprise": "#D87D00","trust": "#026F00"}

  fig_emo = sns.catplot(data = df_melt_emo_pathos_perc.sort_values(by = "value",

                              ascending=False), x = "value", y = "emotion", hue = "emotion",

                              kind = "bar", aspect=1.8, dodge=False, palette = color_map_pathos_emo,

                              height = 6, legend=False)

  plt.xlabel("\npercentage %", fontsize=18)

  plt.ylabel(" ")

  plt.yticks(fontsize=16)

  plt.xticks(np.arange(0, df_melt_emo_pathos_perc["value"].max()+3, 2), fontsize=15)

  plt.xlim(0, df_melt_emo_pathos_perc["value"].max()+3)

  plt.title("Pathos - emotions analytics\n", fontsize=23)

  import matplotlib.patches as mpatches

  red_patch = mpatches.Patch(color="#BB0000", label="negative emotion")

  green_patch = mpatches.Patch(color="#026F00", label="positive emotion")

  gold_patch = mpatches.Patch(color="#D87D00", label="ambivalent emotion")

  plt.legend(handles=[red_patch, green_patch, gold_patch], fontsize=15)

  plt.show()

  return fig_emo
```

plot_pathos_emo_counts(dataframe: dataframe):

Make a barplot with numbers (frequency) of basic emotions.

Full source code:

```
df_melt_emo_pathos = dataframe[pathos_cols[4:]].fillna(0).melt(var_name="emotion",
                                value_name = "value")
df_melt_emo_pathos_perc = df_melt_emo_pathos.groupby(["emotion", "value"],
                                as_index=False).size()
df_melt_emo_pathos_perc = df_melt_emo_pathos_perc[df_melt_emo_pathos_perc.value == 1]
color_map_pathos_emo = {"anger": "#BB0000","anticipation": "#D87D00","disgust": "#BB0000",
                        "fear":"#BB0000","happiness": "#026F00","sadness":
                        "#BB0000","surprise":  "#D87D00","trust": "#026F00"}
fig_emo = sns.catplot(data = df_melt_emo_pathos_perc.sort_values(by = "size", ascending=False),
                        x = "size", y = "emotion", hue = "emotion", kind = "bar", aspect=1.8,
                        dodge=False, palette = color_map_pathos_emo, height = 6, legend=False)
plt.xlabel("\nnumber", fontsize=18)
plt.ylabel(" ")
plt.yticks(fontsize=16)
plt.xticks(np.arange(0, df_melt_emo_pathos_perc["size"].max()+25, 50), fontsize=15)
plt.xlim(0, df_melt_emo_pathos_perc["size"].max()+25)
plt.title("Pathos - emotions analytics\n", fontsize=23)
import matplotlib.patches as mpatches
red_patch = mpatches.Patch(color="#BB0000", label="negative emotion")
green_patch = mpatches.Patch(color="#026F00", label="positive emotion")
gold_patch = mpatches.Patch(color="#D87D00", label="ambivalent emotion")
plt.legend(handles=[red_patch, green_patch, gold_patch], fontsize=15)
plt.show()
return fig_emo
```

plot_rhetoric_basic_stats[1/ 2/ 3]_post(dataframe: dataframe, var_multiselect: list, values_type: str = "percentage"):

First, it transforms a *dataframe* from the ADU unit to the post unit of text. Then, depending on the number of variables in *var_multiselect* (list) — 1, 2, or 3 — makes 1, 2, or 3 plots, respectively (one for each variable).

Related function: *plot_rhetoric_basic_stats[1/ 2/ 3]()*.

Source code below for 2 variables, i.e. *plot_rhetoric_basic_stats2_post()* function.

Full source code:

```
num_vars = len(var_multiselect)
  data = dataframe.reset_index().rename(columns={"index":"Sentence_id"})
  dataframe["Logos_attack"] = np.where(dataframe["ethos_name"] == "attack", 1, 0)
  dataframe["Logos_support"] = np.where(dataframe["ethos_name"] == "support", 1, 0)
  data[["Sentence_id", "Contains_pathos", "positive_valence", "negative_valence",
        "happiness", "anger", "sadness", "fear", "disgust", "surprise", "trust",
        "anticipation", "Contains_ethos", "Support", "Attack", "Ethos_Label"]] = data[["Sentence_id",
        "Contains_pathos", "positive_valence", "negative_valence","happiness", "anger", "sadness",
        "fear", "disgust", "surprise", "trust", "anticipation", "Contains_ethos", "Support", "Attack",
        "Ethos_Label"]].fillna(0).astype(int)
  data["map_ID"] = data["map_ID"].astype(str)
  source = data.Source.unique()
  data1 = data.copy()
  data1["new_id"] = data1["Sentence_id"]
  for i in range(len(data1)-1):
    if (data1.iloc[i+1]["Sentence_id"] == data1.iloc[i]["Sentence_id"]+1) and (data1.iloc[i]["Source"]
        == data1.iloc[i+1]["Source"]) and (data1.iloc[i]["map_ID"] == data1.iloc[i+1]["map_ID"]):
      data1.loc[i+1,"new_id"] = data1.loc[i,"new_id"]
  data1 = data1.astype(str)
  data2 = data1.groupby(["map_ID","Source","new_id"])["map_ID","new_id","Text", "Source",
                                "clean_Text_lemmatized", "Target"].transform(lambda x: ", ".join(x))
  data2 = data2.drop_duplicates()
  for c in ["map_ID","new_id", "Source"]:
    data2[c] = data2[c].apply(lambda x: x.split(", ")[0])
```

```
    data2["Target"] = data2["Target"].apply(lambda x: set(x.split(", ")))
    data2["Target"] = data2["Target"].apply(lambda x: [y for y in x if y != "nan"])
    for c in ["Contains_pathos","positive_valence", "negative_valence", "happiness", "anger",
"sadness",
            "fear", "disgust", "surprise", "trust", "anticipation", "Contains_ethos", "Support",
            "Attack", "Logos_attack", "Logos_support"]:
            data1[c] = data1[c].apply(int)
    data11 = data1.groupby(["map_ID", "Source", "new_id"], as_index=False)["Contains_pathos",
                                        "positive_valence", "negative_valence",
                                        "happiness", "anger", "sadness", "fear", "disgust", "surprise", "trust",
                                        "anticipation", "Contains_ethos", "Support", "Attack", "Logos_attack",
                                         "Logos_support"].sum()
    data11 = data11.drop_duplicates()
    for c in ["Contains_pathos","positive_valence", "negative_valence",
        "happiness", "anger", "sadness", "fear", "disgust", "surprise", "trust",
        "anticipation", "Contains_ethos", "Support", "Attack", "Logos_attack", "Logos_support"]:
          data11[c] = np.where(data11[c] > 0, 1, 0)
    data = data2.merge(data11, on = ["map_ID", "new_id", "Source"])
    var_name1 = var_multiselect[0]
    var_name2 = var_multiselect[1]
    if "ethos" in var_name1:
        att_column1 = "Attack"
        sup_column1 = "Support"
    elif "logos" in var_name1:
        att_column1 = "Logos_attack"
        sup_column1 = "Logos_support"
    elif "pathos" in var_name1:
        att_column1 = "negative_valence"
        sup_column1 = "positive_valence"
    if "ethos" in var_name2:
        att_column2 = "Attack"
        sup_column2 = "Support"
    elif "logos" in var_name2:
```

```
    att_column2 = "Logos_attack"
    sup_column2 = "Logos_support"
elif "pathos" in var_name2:
    att_column2 = "negative_valence"
    sup_column2 = "positive_valence"
if val_type == "number":
    #plot1
    df_prop1a = pd.DataFrame(data[att_column1].value_counts())
    df_prop1a.columns = ["count"]
    df_prop1a.reset_index(inplace=True)
    df_prop1a.columns = ["label", "count"]
    df_prop1a = df_prop1a[df_prop1a.label == 1]
    if "pathos" in var_name1:
        df_prop1a["label"] = df_prop1a["label"].map({1: "negative"})
    else:
        df_prop1a["label"] = df_prop1a["label"].map({1: "attack"})
    df_prop1s = pd.DataFrame(data[sup_column1].value_counts())
    df_prop1s.columns = ["count"]
    df_prop1s.reset_index(inplace=True)
    df_prop1s.columns = ["label", "count"]
    df_prop1s = df_prop1s[df_prop1s.label == 1]
    if "pathos" in var_name1:
        df_prop1s["label"] = df_prop1s["label"].map({1: "positive"})
    else:
        df_prop1s["label"] = df_prop1s["label"].map({1: "support"})
    n_neut = len(data[ (data[sup_column1] == 0) & (data[att_column1] == 0) ])
    df_prop1n = pd.DataFrame({"label": ["neutral"], "count": [n_neut]})
    df_prop1 = pd.concat([df_prop1a, df_prop1s, df_prop1n], axis = 0)
    df_prop1 = df_prop1.reset_index(drop=True)
    df_prop1 = df_prop1.sort_values(by = "label")
    title_str1 = str(var_name1).replace("_name", "").capitalize()
    fig1, ax1 = plt.subplots(figsize=(9, 6))
```

```
ax1.bar(df_prop1["label"], df_prop1["count"], color = ["#BB0000", "#022D96", "#026F00"])
plt.ylabel("number\n", fontsize=16)
plt.yticks(np.arange(0, df_prop1["count"].max()+206, 200), fontsize=15)
plt.xticks(fontsize=17)
plt.title(f"{title_str1} analytics\n", fontsize=23)
vals1 = df_prop1["count"].values
for i, v in enumerate(vals1):
    plt.text(x=i , y = v+1 , s=f"{v}" , fontdict=dict(fontsize=17, ha="center"))
plt.show()

#plot2
df_prop2a = pd.DataFrame(data[att_column2].value_counts())
df_prop2a.columns = ["count"]
df_prop2a.reset_index(inplace=True)
df_prop2a.columns = ["label", "count"]
df_prop2a = df_prop2a[df_prop2a.label == 1]
if "pathos" in var_name2:
    df_prop2a["label"] = df_prop2a["label"].map({1: "negative"})
else:
    df_prop2a["label"] = df_prop2a["label"].map({1: "attack"})
df_prop2s = pd.DataFrame(data[sup_column2].value_counts())
df_prop2s.columns = ["count"]
df_prop2s.reset_index(inplace=True)
df_prop2s.columns = ["label", "count"]
df_prop2s = df_prop2s[df_prop2s.label == 1]
if "pathos" in var_name2:
    df_prop2s["label"] = df_prop2s["label"].map({1: "positive"})
else:
    df_prop2s["label"] = df_prop2s["label"].map({1: "support"})
n_neut2 = len(data[ (data[sup_column2] == 0) & (data[att_column2] == 0) ])
df_prop2n = pd.DataFrame({"label": ["neutral"], "count": [n_neut2]})
df_prop2 = pd.concat([df_prop2a, df_prop2s, df_prop2n], axis = 0)
```

```
    df_prop2 = df_prop2.reset_index(drop=True)
    df_prop2 = df_prop2.sort_values(by = "label")
    title_str2 = str(var_name2).replace("_name", "").capitalize()
    fig2, ax2 = plt.subplots(figsize=(9, 6))
    ax2.bar(df_prop2["label"], df_prop2["count"], color = ["#BB0000", "#022D96", "#026F00"])
    plt.ylabel("number\n", fontsize=16)
    plt.yticks(np.arange(0, df_prop2["count"].max()+206, 200), fontsize=15)
    plt.xticks(fontsize=17)
    plt.title(f"{title_str2} analytics\n", fontsize=23)
    vals2 = df_prop2["count"].values
    for i, v in enumerate(vals2):
        plt.text(x=i , y = v+1 , s=f"{v}" , fontdict=dict(fontsize=17, ha="center"))
    plt.show()
else:
    #plot1
    df_prop1a = pd.DataFrame(data[att_column1].value_counts(normalize=True).round(3)*100)
    df_prop1a.columns = ["percentage"]
    df_prop1a.reset_index(inplace=True)
    df_prop1a.columns = ["label", "percentage"]
    df_prop1a = df_prop1a[df_prop1a.label == 1]
    if "pathos" in var_name1:
        df_prop1a["label"] = df_prop1a["label"].map({1: "negative"})
    else:
        df_prop1a["label"] = df_prop1a["label"].map({1: "attack"})
    df_prop1s = pd.DataFrame(data[sup_column1].value_counts(normalize=True).round(3)*100)
    df_prop1s.columns = ["percentage"]
    df_prop1s.reset_index(inplace=True)
    df_prop1s.columns = ["label", "percentage"]
    df_prop1s = df_prop1s[df_prop1s.label == 1]
    if "pathos" in var_name1:
        df_prop1s["label"] = df_prop1s["label"].map({1: "positive"})
    else:
```

```
    df_prop1s["label"] = df_prop1s["label"].map({1: "support"})
  n_neut = len(data[ (data[sup_column1] == 0) & (data[att_column1] == 0) ])
  n_neut = round(n_neut / len(data) * 100, 1)
  df_prop1n = pd.DataFrame({"label": ["neutral"], "percentage": [n_neut]})
  df_prop1 = pd.concat([df_prop1a, df_prop1s, df_prop1n], axis = 0)
  df_prop1 = df_prop1.reset_index(drop=True)
  df_prop1 = df_prop1.sort_values(by = "label")
  title_str1 = str(var_name1).replace("_name", "").capitalize()
  fig1, ax1 = plt.subplots(2, 1, figsize=(10, 12))
  ax1[0].bar(df_prop1["label"], df_prop1["percentage"], color = ["#BB0000", "#022D96",
"#026F00"])
  ax1[0].set_ylabel("percentage %\n", fontsize=16)
  ax1[0].set_yticks(np.arange(0, df_prop1["percentage"].max()+16, 10), fontsize=15)
  ax1[0].tick_params(axis="x", labelsize=17)
  ax1[0].set_title(f"{title_str1} analytics\n", fontsize=23)
  vals1 = df_prop1["percentage"].values
  for i, v in enumerate(vals1):
    ax1[0].text(x=i , y = v+1 , s=f"{round(v, 1)}%" , fontdict=dict(fontsize=17, ha="center"))
  n_att = len(data[ (data[att_column1] == 1) ])
  n_att = round(n_att / len(data[ (data[att_column1] == 1) | (data[sup_column1] == 1) ])* 100, 1)
  n_sup = len(data[ (data[sup_column1] == 1) ])
  n_sup = round(n_sup / len(data[ (data[att_column1] == 1) | (data[sup_column1] == 1) ])* 100, 1)
  if "pathos" in var_name1:
    df_prop11 = pd.DataFrame({"label": ["negative", "positive"], "percentage": [n_att, n_sup]})
  else:
    df_prop11 = pd.DataFrame({"label": ["attack", "support"], "percentage": [n_att, n_sup]})
  df_prop11 = df_prop11.sort_values(by = "label")
  ax1[1].bar(df_prop11["label"], df_prop11["percentage"], color = ["#BB0000", "#026F00"])
  ax1[1].set_ylabel("percentage %\n", fontsize=16)
  ax1[1].set_yticks(np.arange(0, df_prop11["percentage"].max()+11, 10), fontsize=15)
  ax1[1].tick_params(axis="x", labelsize=17)
  ax1[1].set_title(f"\n", fontsize=23)
  vals11 = df_prop11["percentage"].values
```

```
for i, v in enumerate(vals11):
    ax1[1].text(x=i , y = v+1 , s=f"{round(v, 0)}%" , fontdict=dict(fontsize=17, ha="center"))
plt.tight_layout()
plt.show()

#plot2
df_prop2a = pd.DataFrame(data[att_column2].value_counts(normalize=True).round(3)*100)
df_prop2a.columns = ["percentage"]
df_prop2a.reset_index(inplace=True)
df_prop2a.columns = ["label", "percentage"]
df_prop2a = df_prop2a[df_prop2a.label == 1]
if "pathos" in var_name2:
    df_prop2a["label"] = df_prop2a["label"].map({1: "negative"})
else:
    df_prop2a["label"] = df_prop2a["label"].map({1: "attack"})
df_prop2s = pd.DataFrame(data[sup_column2].value_counts(normalize=True).round(3)*100)
df_prop2s.columns = ["percentage"]
df_prop2s.reset_index(inplace=True)
df_prop2s.columns = ["label", "percentage"]
df_prop2s = df_prop2s[df_prop2s.label == 1]
if "pathos" in var_name2:
    df_prop2s["label"] = df_prop2s["label"].map({1: "positive"})
else:
    df_prop2s["label"] = df_prop2s["label"].map({1: "support"})
n_neut2 = len(data[ (data[sup_column2] == 0) & (data[att_column2] == 0) ])
n_neut2 = round(n_neut2 / len(data) * 100, 1)
df_prop2n = pd.DataFrame({"label": ["neutral"], "percentage": [n_neut2]})
df_prop2 = pd.concat([df_prop2a, df_prop2s, df_prop2n], axis = 0)
df_prop2 = df_prop2.reset_index(drop=True)
df_prop2 = df_prop2.sort_values(by = "label")
title_str2 = str(var_name2).replace("_name", "").capitalize()
fig2, ax2 = plt.subplots(2, 1, figsize=(10, 12))
```

```
    ax2[0].bar(df_prop2["label"], df_prop2["percentage"], color = ["#BB0000", "#022D96",
                    "#026F00"])
    ax2[0].set_ylabel("percentage %\n", fontsize=16)
    ax2[0].set_yticks(np.arange(0, df_prop2["percentage"].max()+16, 10), fontsize=15)
    ax2[0].tick_params(axis="x", labelsize=17)
    ax2[0].set_title(f"{title_str2} analytics\n", fontsize=23)
    vals2 = df_prop2["percentage"].values
    for i, v in enumerate(vals2):
        ax2[0].text(x=i , y = v+1 , s=f"{round(v, 1)}%" , fontdict=dict(fontsize=17, ha="center"))
    n_att = len(data[ (data[att_column2] == 1) ])
    n_att = round(n_att / len(data[ (data[att_column2] == 1) | (data[sup_column2] == 1) ])* 100, 1)
    n_sup = len(data[ (data[sup_column2] == 1) ])
    n_sup = round(n_sup / len(data[ (data[att_column2] == 1) | (data[sup_column2] == 1) ])* 100, 1)
    if "pathos" in var_name2:
        df_prop21 = pd.DataFrame({"label": ["negative", "positive"], "percentage": [n_att, n_sup]})
    else:
        df_prop21 = pd.DataFrame({"label": ["attack", "support"], "percentage": [n_att, n_sup]})
    df_prop21 = df_prop21.sort_values(by = "label")
    ax2[1].bar(df_prop21["label"], df_prop21["percentage"], color = ["#BB0000", "#026F00"])
    ax2[1].set_ylabel("percentage %\n", fontsize=16)
    ax2[1].set_yticks(np.arange(0, df_prop21["percentage"].max()+11, 10), fontsize=15)
    ax2[1].tick_params(axis="x", labelsize=17)
    ax2[1].set_title(f"\n", fontsize=23)
    vals22 = df_prop21["percentage"].values
    for i, v in enumerate(vals22):
        ax2[1].text(x=i , y = v+1 , s=f"{round(v, 0)}%" , fontdict=dict(fontsize=17, ha="center"))
    plt.tight_layout()
    plt.show()
return fig1, fig2
```

plot_rhetoric_basic_stats[1/ 2/ 3](dataframe: dataframe, var_multiselect: list, values_type: str = "percentage"):

Depending on the number of variables in *var_multiselect* (list) — 1, 2, or 3 — makes 1, 2, or 3 plots, respectively (one for each variable).

Related function: *plot_rhetoric_basic_stats[1/ 2/ 3]_post()*.

Source code below for 2 variables, i.e. *plot_rhetoric_basic_stats2()* function.

Full source code:

```
num_vars = len(var_multiselect)
  if val_type == "number":
    #plot1
    var_name1 = var_multiselect[0]
    df_prop1 = pd.DataFrame(dataframe[str(var_name1)].value_counts())
    df_prop1.columns = ["count"]
    df_prop1.reset_index(inplace=True)
    df_prop1.columns = ["label", "count"]
    #df_prop1["label"] = df_prop1["label"].str.replace("negative", " negative")
    df_prop1 = df_prop1.sort_values(by = "label")
    title_str1 = str(var_name1).replace("_name", "").capitalize()
    fig1, ax1 = plt.subplots(figsize=(9, 6))
    ax1.bar(df_prop1["label"], df_prop1["count"], color = ["#BB0000", "#022D96", "#026F00"])
    plt.ylabel("number\n", fontsize=16)
    plt.yticks(np.arange(0, df_prop1["count"].max()+206, 500), fontsize=15)
    plt.xticks(fontsize=17)
    plt.title(f"{title_str1} analytics\n", fontsize=23)
    vals1 = df_prop1["count"].values
    for i, v in enumerate(vals1):
      plt.text(x=i , y = v+1 , s=f"{v}" , fontdict=dict(fontsize=17, ha="center"))
    plt.show()
    #plot2
    var_name2 = var_multiselect[1]
    df_prop2 = pd.DataFrame(dataframe[str(var_name2)].value_counts())
```

```
    df_prop2.columns = ["count"]
    df_prop2.reset_index(inplace=True)
    df_prop2.columns = ["label", "count"]
    #df_prop2["label"] = df_prop2["label"].str.replace("negative", " negative")
    df_prop2 = df_prop2.sort_values(by = "label")
    title_str2 = str(var_name2).replace("_name", "").capitalize()
    fig2, ax2 = plt.subplots(figsize=(9, 6))
    ax2.bar(df_prop2["label"], df_prop2["count"], color = ["#BB0000", "#022D96", "#026F00"])
    plt.ylabel("number\n", fontsize=16)
    plt.yticks(np.arange(0, df_prop2["count"].max()+206, 500), fontsize=15)
    plt.xticks(fontsize=17)
    plt.title(f"{title_str2} analytics\n", fontsize=23)
    vals2 = df_prop2["count"].values
    for i, v in enumerate(vals2):
        plt.text(x=i , y = v+1 , s=f"{v}" , fontdict=dict(fontsize=17, ha="center"))
    plt.show()
else:
    #plot1
    var_name1 = var_multiselect[0]
    df_prop1 =
                pd.DataFrame(dataframe[str(var_name1)].value_counts(normalize=True).round(3)*100)
    df_prop1.columns = ["percentage"]
    df_prop1.reset_index(inplace=True)
    df_prop1.columns = ["label", "percentage"]
    #df_prop1["label"] = df_prop1["label"].str.replace("negative", " negative")
    df_prop1 = df_prop1.sort_values(by = "label")
    title_str1 = str(var_name1).replace("_name", "").capitalize()
    fig1, ax1 = plt.subplots(2, 1, figsize=(10, 12))
    ax1[0].bar(df_prop1["label"], df_prop1["percentage"],
                    color = ["#BB0000", "#022D96", "#026F00"])
    ax1[0].set_ylabel("percentage\n", fontsize=16)
    ax1[0].set_yticks(np.arange(0, df_prop1["percentage"].max()+16, 10), fontsize=15)
```

```
ax1[0].tick_params(axis="x", labelsize=17)
ax1[0].set_title(f"{title_str1} analytics\n", fontsize=23)
vals1 = df_prop1["percentage"].values
for i, v in enumerate(vals1):
   ax1[0].text(x=i , y = v+1 , s=f"{round(v, 1)}%" , fontdict=dict(fontsize=17, ha="center"))
df_prop11 = pd.DataFrame(dataframe[dataframe[str(var_name1)] !=
                "neutral"][str(var_name1)].value_counts(normalize=True).round(3)*100)
df_prop11.columns = ["percentage"]
df_prop11.reset_index(inplace=True)
df_prop11.columns = ["label", "percentage"]
df_prop11 = df_prop11.sort_values(by = "label")
ax1[1].bar(df_prop11["label"], df_prop11["percentage"], color = ["#BB0000", "#026F00"])
ax1[1].set_ylabel("percentage\n", fontsize=16)
ax1[1].set_yticks(np.arange(0, df_prop11["percentage"].max()+11, 10), fontsize=15)
ax1[1].tick_params(axis="x", labelsize=17)
ax1[1].set_title(f"\n", fontsize=23)
vals11 = df_prop11["percentage"].values
for i, v in enumerate(vals11):
   ax1[1].text(x=i , y = v+1 , s=f"{round(v, 0)}%" , fontdict=dict(fontsize=17, ha="center"))
plt.tight_layout()
plt.show()

#plot2
var_name2 = var_multiselect[1]
df_prop2 =
            pd.DataFrame(dataframe[str(var_name2)].value_counts(normalize=True).round(3)*100)
df_prop2.columns = ["percentage"]
df_prop2.reset_index(inplace=True)
df_prop2.columns = ["label", "percentage"]
#df_prop2["label"] = df_prop2["label"].str.replace("negative", " negative")
df_prop2 = df_prop2.sort_values(by = "label")
title_str2 = str(var_name2).replace("_name", "").capitalize()
```

```
    fig2, ax2 = plt.subplots(2, 1, figsize=(10, 12))
    ax2[0].bar(df_prop2["label"], df_prop2["percentage"], color = ["#BB0000", "#022D96",
"#026F00"])
    ax2[0].set_ylabel("percentage\n", fontsize=16)
    ax2[0].set_yticks(np.arange(0, df_prop2["percentage"].max()+16, 10), fontsize=15)
    ax2[0].tick_params(axis="x", labelsize=17)
    ax2[0].set_title(f"{title_str2} analytics\n", fontsize=23)
    vals2 = df_prop2["percentage"].values
    for i, v in enumerate(vals2):
        ax2[0].text(x=i , y = v+1 , s=f"{round(v, 1)}%" , fontdict=dict(fontsize=17, ha="center"))
    df_prop22 = pd.DataFrame(dataframe[dataframe[str(var_name2)] !=
                    "neutral"][str(var_name2)].value_counts(normalize=True).round(3)*100)
    df_prop22.columns = ["percentage"]
    df_prop22.reset_index(inplace=True)
    df_prop22.columns = ["label", "percentage"]
    df_prop22 = df_prop22.sort_values(by = "label")
    ax2[1].bar(df_prop22["label"], df_prop22["percentage"], color = ["#BB0000", "#026F00"])
    ax2[1].set_ylabel("percentage\n", fontsize=16)
    ax2[1].set_yticks(np.arange(0, df_prop22["percentage"].max()+11, 10), fontsize=15)
    ax2[1].tick_params(axis="x", labelsize=17)
    ax2[1].set_title(f"\n", fontsize=23)
    vals22 = df_prop22["percentage"].values
    for i, v in enumerate(vals22):
        ax2[1].text(x=i , y = v+1 , s=f"{round(v, 0)}%" , fontdict=dict(fontsize=17, ha="center"))
    plt.tight_layout()
    plt.show()
  return fig1, fig2
```

ethos_logos_LEPbehavior_metric(dataframe: dataframe, source_column: str = "Source", labels_column: str = "ethos_name"):

Calculates "rhetoric" behaviour for each user on a single category (logos or ethos). It multiplies the weights (map_weight**) and number of texts from each category of attack, neutral, and support.**

Related function: *user_rhetoric_v2()* and *pathos_LEPbehavior_metric()* .

Full source code:

```
map_weight = {
        "attack":-1,
        "neutral":0,
        "support":1      }
 ethos_metric_value = []
 users_list = []
 for user in dataframe:
  users_list.append(user)
  df_user = dataframe[dataframe[source_column] == user]
  df_user_ethos = df_user.groupby(labels_column, as_index = False)["Text"].size()
  if ("support" in df_user_ethos[labels_column].unique()) or ("attack" in
        df_user_ethos[labels_column].unique()):
   if "support" in df_user_ethos[labels_column].unique():
    Nsup_u = df_user_ethos[df_user_ethos[labels_column] == "support"]["size"].iloc[0]
   else:
    Nsup_u = 0
   if "attack" in df_user_ethos[labels_column].unique():
    Natt_u = df_user_ethos[df_user_ethos[labels_column] == "attack"]["size"].iloc[0]
   else:
    Natt_u = 0
   ethos_user_support = 1 * int(Nsup_u)
   ethos_user_attack = -1 * int(Natt_u)
   ethos_user = (ethos_user_support) + (ethos_user_attack)
   ethos_metric_value.append(ethos_user)
  else:
```

```
    ethos_metric_value.append(0)
df_metric = pd.DataFrame({"source": users_list, "rhetoric_metric": ethos_metric_value})
return df_metric
```

pathos_LEPbehavior_metric(dataframe: dataframe, source_column: str = "Source", labels_column: str = "pathos_name"):

Calculates LEP behaviour for each user on a single category (pathos). It multiplies the weights (map_weight**) and number of texts from each category of negative, neutral, and positive pathos.**

Related function: *user_rhetoric_v2()* and *ethos_logos_LEPbehavior_metric()*.

Full source code:

```
map_weight = {
        "negative":-1,
        "neutral":0,
        "positive":1      }
 pathos_metric_value = []
 users_list = []
 for user in dataframe:
  users_list.append(user)
  df_user = dataframe[dataframe[source_column] == user]
  df_user_pathos = df_user.groupby(labels_column, as_index = False)["Text"].size()
  if ("positive" in df_user_pathos[labels_column].unique()) or ("negative" in
        df_user_pathos[labels_column].unique()):
   if "positive" in df_user_pathos[labels_column].unique():
    Nsup_u = df_user_pathos[df_user_pathos[labels_column] == "positive"]["size"].iloc[0]
   else:
    Nsup_u = 0
   if "negative" in df_user_pathos[labels_column].unique():
    Natt_u = df_user_pathos[df_user_pathos[labels_column] == "negative"]["size"].iloc[0]
   else:
    Natt_u = 0
   pathos_user_support = 1 * int(Nsup_u)
   pathos_user_attack = -1 * int(Natt_u)
   pathos_user = (pathos_user_support) + (pathos_user_attack)
```

```
    pathos_metric_value.append(pathos_user)
  else:
    pathos_metric_value.append(0)
df_metric = pd.DataFrame({"source": users_list, "rhetoric_metric": pathos_metric_value})
return df_metric
```

plotRhetoricCompare[2/ 3](dataframe1, dataframe2, (dataframe3), rhetoric_dimension: str):

Visualise in one figure distributions of the category specified in *rhetoric_dimension* (logos, ethos, or pathos) for all dataframes.

Related function: *plot_rhetoric_basic_stats[1/ 2/ 3]()*.

Source code below for 2 variables, i.e. *plotRhetoricCompare2()* function.

Full source code:

```
rhet_dim = str(rhet_dim_to_plot)
  rhet_dim_var = rhet_dim.replace("ethos", "ethos_name").replace("logos",
                        "logos_name").replace("pathos", "pathos_name")
  df_prop1 = pd.DataFrame(data1[rhet_dim_var].value_counts(normalize=True).round(3)*100)
  df_prop1.columns = ["percentage"]
  df_prop1.reset_index(inplace=True)
  df_prop1.columns = ["label", "percentage"]
  df_prop1 = df_prop1.sort_values(by = "label")
  df_prop2 = pd.DataFrame(data2[rhet_dim_var].value_counts(normalize=True).round(3)*100)
  df_prop2.columns = ["percentage"]
  df_prop2.reset_index(inplace=True)
  df_prop2.columns = ["label", "percentage"]
  df_prop2 = df_prop2.sort_values(by = "label")
  fig_stats, axs = plt.subplots(1, 2, figsize=(18, 7), sharey=True, constrained_layout=True)
  axs[0].bar(df_prop1["label"], df_prop1["percentage"], color = ["#BB0000", "#022D96", "#026F00"])
  title_str0 = str(rhet_dim).capitalize() + " in " + f"*{data1["Dataset"].iloc[0]}*"
  axs[0].set_title(title_str0, fontsize=20)
  axs[0].set_ylabel("percentage %\n", fontsize=16)
  axs[0].set_ylim(0, 101)
  axs[0].tick_params(axis="x", labelsize=17)
  axs[0].tick_params(axis="y", labelsize=15)
  vals0 = df_prop1["percentage"].values.round(1)
  for i, v in enumerate(vals0):
    axs[0].text(x=i , y = v+1 , s=f"{v}%" , fontdict=dict(fontsize=14, ha="center", weight="bold"))
```

```
axs[1].bar(df_prop2["label"], df_prop2["percentage"], color = ["#BB0000", "#022D96", "#026F00"])
title_str1 = str(rhet_dim).capitalize() + " in " + f"*{data2["Dataset"].iloc[0]}*"
axs[1].set_title(title_str1, fontsize=20)
#axs[1].set_ylabel("percentage %\n", fontsize=16)
axs[1].yaxis.set_tick_params(labelbottom=True)
axs[1].tick_params(axis="x", labelsize=17)
axs[1].tick_params(axis="y", labelsize=15)
vals1 = df_prop2["percentage"].values.round(1)
for i, v in enumerate(vals1):
    axs[1].text(x=i , y = v+1 , s=f"{v}%" , fontdict=dict(fontsize=14, ha="center", weight="bold"))
fig_stats.subplots_adjust(hspace=.3)
plt.show()
st.pyplot(fig_stats)
```

plotRhetoricCompare[2/ 3]_finegrained(dataframe1, dataframe2, (dataframe3), rhetoric_dimension: str):

Visualise in one figure distributions of the category specified in *rhetoric_dimension* (logos, ethos, or pathos) for all dataframes. However, it does not include the "neutral" category so only proportions for support vs. attack, or positive vs. negative categories are depicted.

Related function: *plot_rhetoric_basic_stats[1/ 2/ 3]()*.

Source code below for 2 variables, i.e. *plotRhetoricCompare2_finegrained()* function.

Full source code:

```
if rhet_dim_to_plot == "logos":
    rhet_dim = str(rhet_dim_to_plot)
    rhet_dim_var = str(rhet_dim_to_plot)
    data1[rhet_dim_var] = data1[rhet_dim_var].map({"CA":"attack", "RA":"support"})
    data2[rhet_dim_var] = data2[rhet_dim_var].map({"CA":"attack", "RA":"support"})
  else:
    rhet_dim = str(rhet_dim_to_plot)
    rhet_dim_var = rhet_dim.replace("ethos", "ethos_name").replace("logos",
                                "logos_name").replace("pathos", "pathos_name")
  data1 = data1[data1[rhet_dim_var] != "neutral"]
  df_prop1 = pd.DataFrame(data1[rhet_dim_var].value_counts(normalize=True).round(3)*100)
  df_prop1.columns = ["percentage"]
  df_prop1.reset_index(inplace=True)
  df_prop1.columns = ["label", "percentage"]
  df_prop1 = df_prop1.sort_values(by = "label")
  if hansard_box and radio_targets == "3rd party":
    df_prop1["percentage"] = [0, 0]
  data2 = data2[data2[rhet_dim_var] != "neutral"]
  df_prop2 = pd.DataFrame(data2[rhet_dim_var].value_counts(normalize=True).round(3)*100)
  df_prop2.columns = ["percentage"]
  df_prop2.reset_index(inplace=True)
  df_prop2.columns = ["label", "percentage"]
  df_prop2 = df_prop2.sort_values(by = "label")
```

```
fig_stats2, axs = plt.subplots(1, 2, figsize=(18, 7), sharey=True, constrained_layout=True)
axs[0].bar(df_prop1["label"], df_prop1["percentage"], color = ["#BB0000", "#026F00"])
title_str0 = str(rhet_dim).capitalize() + " in " + f"*{data1["Dataset"].iloc[0]}*"
axs[0].set_title(title_str0, fontsize=20)
axs[0].set_ylabel("percentage %\n", fontsize=16)
axs[0].set_ylim(0, 101)
axs[0].tick_params(axis="x", labelsize=17)
axs[0].tick_params(axis="y", labelsize=15)
vals0 = df_prop1["percentage"].values.round(0)
for i, v in enumerate(vals0):
    axs[0].text(x=i , y = v+1 , s=f"{v}%" , fontdict=dict(fontsize=14, ha="center", weight="bold"))
axs[1].bar(df_prop2["label"], df_prop2["percentage"], color = ["#BB0000", "#026F00"])
title_str1 = str(rhet_dim).capitalize() + " in " + f"*{data2["Dataset"].iloc[0]}*"
axs[1].set_title(title_str1, fontsize=20)
axs[1].yaxis.set_tick_params(labelbottom=True)
axs[1].tick_params(axis="x", labelsize=17)
axs[1].tick_params(axis="y", labelsize=15)
vals1 = df_prop2["percentage"].values.round(0)
for i, v in enumerate(vals1):
    axs[1].text(x=i , y = v+1 , s=f"{v}%" , fontdict=dict(fontsize=14, ha="center", weight="bold"))
fig_stats2.subplots_adjust(hspace=.3)
plt.show()
st.pyplot(fig_stats2)
```